H1(선택적 토큰 재계산) 실험 보고서

온디바이스 AI 4조 | 작성: 장준용 | 2026. 11. 3.

환경: RTX 4090, Core i5-13400 (문화관 408호) | 데이터: ImageNet-1K 검증 이미지 50,000장

한 줄 결론: H1은 고르지 않은 토큰을 '그대로 두는' 대신 '빼고' 파인튜닝하면 같은 연산량에서 기존 방법보다 정확해집니다. 이 효과는 CPU로 돌리는 온디바이스 환경에서 실제 시간 절약으로 이어집니다.

0. 출발점

ThinkingViT는 이미지를 먼저 작은 모델(3H)로 보고, 답에 확신이 없을 때만 큰 모델(6H)로 다시 봅니다. 기준값 τ = 1.0이면 50,000장 중 36%(18,004장)가 6H로 넘어가고 전체 정확도는 80.30%입니다. 이하 6H로 넘어간 이미지를 '어려운 이미지'라고 부릅니다. ViT는 이미지를 조각 196개로 나누고, 조각마다 만든 토큰을 12개 층에서 계산합니다.

팀의 H1 아이디어는 "6H에서 토큰 196개를 다 다시 계산하지 말고, 3H가 많이 주목한 토큰만 골라 계산하자"였습니다. 고르지 않은 토큰은 계산하지 않고 그대로 둡니다(이하 '기존 H1 방식'). 그런데 75%를 계산해도 전체 정확도가 57.8%로, 6H를 안 쓰고 3H만 쓸 때(73.5%)보다 낮았습니다. 어려운 이미지는 50%를 계산했을 때 0.2%였는데, 1,000개 클래스 중 찍어도 0.1%입니다. 그래서 이건 '조금 손해'가 아니라 '고장'이라고 봤습니다.

1. 왜 망가졌나 (원인 분석)

- **가설**: 계산하지 않은 토큰을 그냥 두면 그 토큰은 처음 상태 그대로 끝까지 남습니다. 다른 토큰들이 층마다 이 낡은 값을 참고하면서 계산이 틀어진다고 봤습니다. 다른 가능성은 중요한 토큰을 덜 계산해서 정보가 부족한 것입니다.
- **실험**: 계산 안 할 토큰을 그대로 두지 않고 6H에서 아예 제거해 봤습니다(이하 '토큰 제거 방식'). 정확도가 돌아오면 낡은 토큰이 문제이고, 그대로면 정보 부족이 문제입니다.
- **결과**: 50% 계산 기준, 어려운 이미지 정확도가 0.2%에서 58.0%로 돌아왔습니다(전부 계산하면 62.7%). 원인은 정보 부족이 아니라 남아 있는 토큰의 방해였습니다.

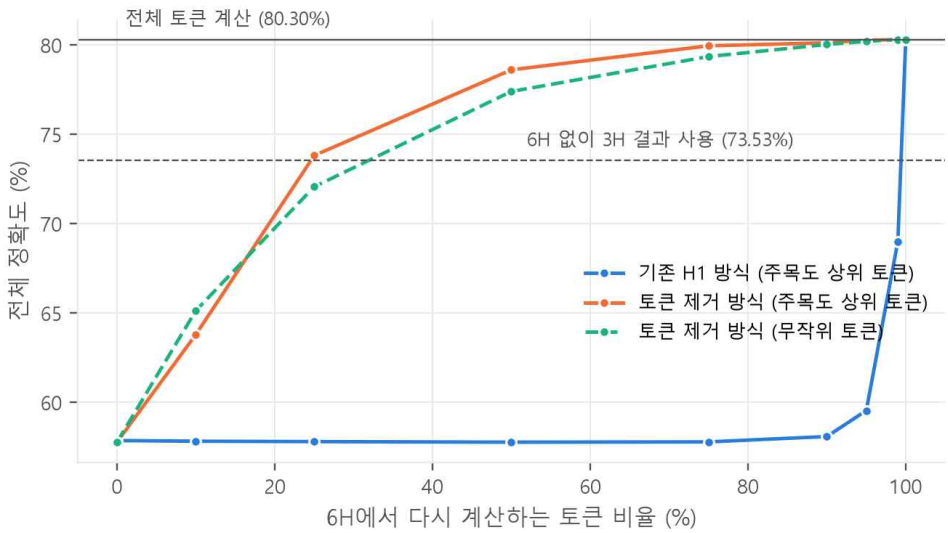


그림 1. 다시 계산하는 토큰 비율에 따른 전체 정확도

표 1. 어려운 이미지 18,004장의 정확도 (3H가 많이 주목한 토큰부터 선택)

다시 계산하는 비율	10%	25%	50%	75%	90%	99%
기존 H1 방식	0.3%	0.3%	0.2%	0.2%	1.0%	31.4%
토큰 제거 방식	16.9%	44.7%	58.0%	61.8%	62.2%	62.7%

- **더 알게 된 것**: 토큰을 고르는 기준도 중요합니다. 50%에서 3H가 많이 주목한 토큰을 고르면 58.0%, 무작위로 고르면 54.7%, 적게 주목한 토큰을 고르면 32.0%였습니다. 25% 이하로 줄이면 토큰 제거 방식도 많이 떨어지는데, 이 구간에서는 정보 부족도 작용합니다. 6H의 앞쪽 8~10층을 모든 토큰으로 계산하면 기존 H1 방식도 회복되지만(57.9~62.7%), 그만큼 아끼는 연산이 적습니다.

2. 기존 방법보다 나은가 (같은 연산량 비교)

- **비교 대상:** 연산을 줄이는 가장 쉬운 방법은 τ를 키워 6H로 보내는 이미지 수를 줄이는 것입니다(이하 '임계값 조정'). 새 방법은 같은 연산량에서 이보다 정확해야 의미가 있습니다. 연산량은 이미지 한 장당 평균 곱셈-덧셈 횟수(GMACs)로 셌습니다. 3H는 1.25, 6H는 4.61입니다.

표 2. 같은 연산량에서 비교 (추가 학습 없음)

방법	평균 GMACs	전체 정확도	같은 연산량의 임계값 조정	차이
원래 모델 (τ = 1.0)	2.92	80.30%	80.30%	0
토큰 제거 25%	1.67	73.81%	75.99%	-2.18%p
토큰 제거 50%	2.07	78.61%	77.89%	+0.72%p
토큰 제거 75%	2.48	79.95%	79.30%	+0.65%p
토큰 제거 50% (무작위로 고름)	2.07	77.40%	77.89%	-0.49%p
기존 H1 방식 50%	2.23	57.77%	78.48%	-20.71%p

- **결과:** 토큰 제거 50~75%가 같은 연산량에서 임계값 조정보다 0.65~0.72%p 높았습니다. 같은 정확도로 맞추면 연산이 약 9% 적습니다. 무작위로 고르거나 기존 H1 방식을 쓰면 더 낮았습니다.
- **우연인지 확인:** 여러 비율 중 제일 좋은 것만 보고하면 운 좋게 잘 나온 값을 고를 수 있습니다. 그래서 이미지를 반씩 나눠 한쪽에서 비율을 고르고 다른 쪽에서 채점하기를 200번 했습니다. 평균 +0.68%p였고 200번 모두 임계값 조정보다 높았습니다.

3. 파인튜닝 (train 이미지 128만 장, 약 50분)

- **왜 필요한가:** 모델은 토큰 197개가 다 있는 상태로만 학습됐습니다. 토큰을 뺀 상태에 익숙해지도록 조금 더 학습시키는 것이 파인튜닝입니다. 검증 이미지는 채점용이라 학습용 train 이미지 128만 장을 따로 받았습니다.
- **방법:** 원래 모델(교사)의 답을, 토큰을 뺀 모델(학생)이 따라 하도록 학습시켰습니다(지식 증류). 3H는 6H와 가중치를 같이 쓰기 때문에 3H의 답도 원래대로 유지되게 함께 학습했습니다. 3에폭 학습했고, 남기는 토큰 비율은 25%와 50%를 섞었습니다. 학습 중 PC가 재부팅돼 중간 저장본에서 이어서 학습했습니다.

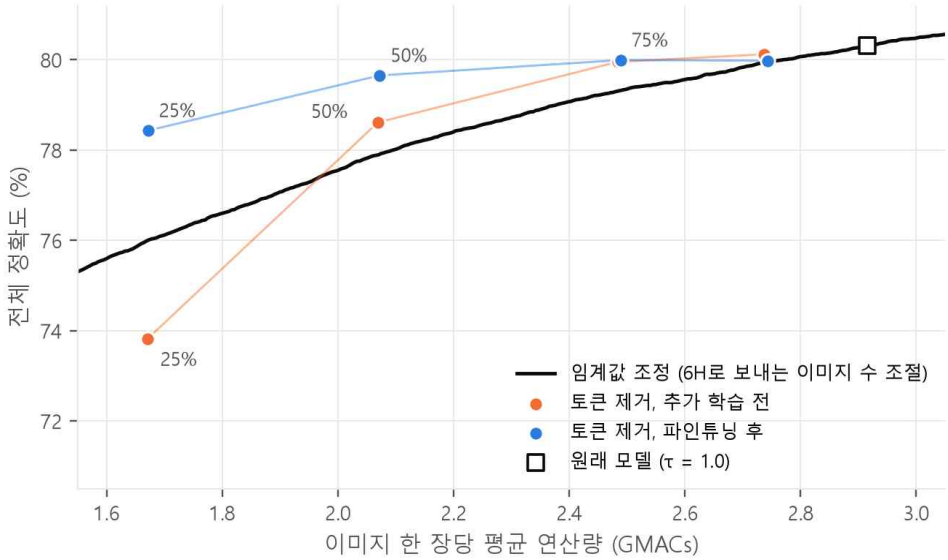


그림 2. 연산량과 정확도 (검은 선보다 위에 있으면 같은 연산량에서 더 정확함)

표 3. 파인튜닝 전후 비교

다시 계산하는 비율	25%	50%	75%
어려운 이미지 정확도 (전 → 후)	44.7% → 57.5%	58.0% → 60.9%	61.8% → 61.8%
전체 정확도 (전 → 후)	73.81% → 78.43%	78.61% → 79.65%	79.95% → 79.99%
같은 연산량의 임계값 조정	76.00%	77.91%	79.32%
임계값 조정과의 차이	+2.43%p	+1.74%p	+0.67%p
같은 정확도에서 줄어드는 연산량	24%	21%	10%

- **결과:** 25%만 계산해도 정확도가 73.81%에서 78.43%가 됐습니다. 같은 연산량의 임계값 조정보다 2.4%p 높고, 같은 정확도를 내는 데 연산이 24% 적습니다. 50%도 1.7%p 높습니다.
- **대가:** 토큰을 전부 계산할 때의 정확도는 81.44%에서 81.03%로 0.4%p 내려갔습니다. 3H 정확도는 그대로입니다(73.53% → 73.60%).

4. 실제로 빨라지나 (추론 시간)

- **측정 방법:** 검증 이미지 1,000장을 한 장씩 넣고, 3H 계산부터 6H 계산까지 이미지 한 장의 전체 추론 시간을 쟀습니다. 휴대폰처럼 코어가 적은 경우를 보려고 CPU는 1코어와 4코어로 나눠 쟀습니다. 이 1,000장 중 6H로 넘어간 비율은 33%입니다.

표 4. 방법별 정확도와 이미지 한 장당 평균 추론 시간 (τ = 1.0, 단위 ms)

방법	전체 정확도	평균 GMACs	GPU	CPU 4코어	CPU 1코어
3H만 사용	73.53%	1.25	4.0	11.3	28.0
원래 모델 (τ = 1.0)	80.30%	2.92	5.5	20.4	56.6
모든 이미지를 6H까지	81.44%	5.87	8.7	38.6	114.0
기존 H1 방식 50%	57.77%	2.23*	5.7	20.6	57.0
토큰 제거 25% (파인튜닝 후)	78.43%	1.67	5.7	15.7	38.2
토큰 제거 50% (파인튜닝 후)	79.65%	2.07	5.7	17.1	44.1
토큰 제거 75% (파인튜닝 후)	79.99%	2.48	5.7	18.8	50.1

* 이론값입니다. 지금 구현은 모든 토큰을 계산한 뒤 결과를 버리므로 실제 연산량은 원래 모델과 같습니다. 토큰 제거 방식의 추론 시간은 파인튜닝 전후가 같습니다.

- **CPU:** 줄인 만큼 빨라집니다. 1코어에서 이미지 한 장의 추론 시간이 56.6ms에서 38.2ms(토큰 제거 25%)로 32% 줄었고, 4코어에서는 20.4ms에서 15.7ms로 23% 줄었습니다. 토큰 제거 50%도 1코어에서 22% 줄었습니다.
- **GPU:** 한 장씩 처리하면 빨라지지 않습니다. GPU는 많은 계산을 동시에 하기 때문에 토큰 수가 줄어도 시간이 거의 같고, 토큰을 고르는 과정만 늘어납니다. 여러 장을 묶어서 처리하는 경우는 재지 않았습니다.
- **기존 H1 방식:** 지금 구현은 모든 토큰을 계산한 뒤 결과를 버리는 방식이라 시간이 전혀 줄지 않습니다.

5. 한계와 다음 할 일

- **한계:** 추론 시간은 이미지를 한 장씩 넣을 때만 쟀고, 실제 휴대폰에서는 재지 않았습니다. 모델 하나(ThinkingViT)와 데이터셋 하나(ImageNet)만 썼고, 파인튜닝은 한 번만 했습니다.
- **다음 할 일:** 휴대폰에서 실제 시간을 재고, 10% 이하로 줄일 때 생기는 정보 부족 문제를 줄여 볼 계획입니다.

코드와 결과: github.com/hee4040/OnDevice (experiment/h1-diagnosis 브랜치, baseline/h1_diagnosis.md)